

AI Navigation Assistant

for Visually Impaired People

Product Requirements Document + Claude Prompt Engineering Guide

Version 1.0 | Laptop + Webcam Prototype | English + Hindi Voice

Model: *claude-opus-4-6* | Stack: *Python · YOLOv8 · MiDaS · ByteTrack · OpenCV*

Target Hardware	Laptop + Webcam (Basic Prototype)
Voice Languages	English + Hindi (Bilingual)
AI Model	claude-opus-4-6 (Anthropic)
Detection	YOLOv8n / YOLOv8s (Ultralytics)
Depth Estimation	MiDaS v2.1 Small
Object Tracking	ByteTrack (via Ultralytics)
Target Latency	< 100ms end-to-end
Deployment	Desktop Demo (laptop)

Table of Contents

1. Product Overview
2. Technology Stack
3. System Modules — Detailed
4. Hallucination Prevention Strategy
5. Performance Requirements
6. Project Folder Structure
7. Claude claude-opus-4-6 Prompt Engineering Guide
 - 7.1 System Prompt
 - 7.2 Scene Description
 - 7.3 Hindi Translation
 - 7.4 Emergency Stop Alert
 - 7.5 Multi-Object Priority
 - 7.6 Stair Detection
 - 7.7 Clear Path Confirmation
 - 7.8 Startup Greeting
 - 7.9 Dynamic Instruction Generator
 - 7.10 Object Distance Narration
 - 7.11 Environment Type Classifier
 - 7.12 Danger Level Assessor
8. Prompt Variables Reference
9. Development Phases
10. Python Requirements

1. Product Overview

A real-time AI-powered navigation assistant that uses a laptop webcam to detect objects, estimate distances, and deliver bilingual (English + Hindi) voice guidance to visually impaired users. Built using Python, YOLOv8, MiDaS depth estimation, and ByteTrack — optimized for low-latency desktop prototype deployment.

1.1 Core Goals

- Detect obstacles and objects in real time from webcam feed
- Estimate object distance using monocular depth (MiDaS)
- Determine LEFT / CENTER / RIGHT position of each detected object
- Track objects across frames to avoid repeated announcements
- Generate voice alerts in both English and Hindi
- Keep total pipeline latency under 100ms
- Prevent hallucinations via confidence thresholding + multi-frame verification
- Integrate Claude claude-opus-4-6 for high-level scene understanding on demand

1.2 Out of Scope (v1 Prototype)

- GPS / outdoor navigation
- Traffic light detection
- Face recognition
- Mobile / Jetson deployment
- Internet-dependent real-time mapping

2. Technology Stack

Component	Technology	Purpose
Object Detection	YOLOv8n / YOLOv8s (Ultralytics)	Real-time multi-object detection
Depth Estimation	MiDaS v2.1 Small	Monocular distance estimation from 2D frame
Object Tracking	ByteTrack (Ultralytics built-in)	Persistent object IDs across frames
Video Capture	OpenCV (cv2)	Webcam frame capture & preprocessing
Voice Output (EN)	pyttsx3 / edge-tts	English TTS, fully offline capable
Voice Output (HI)	gTTS + playsound	Hindi TTS (online, very fast)
AI Scene Understanding	Claude claude-opus-4-6 (Anthropic API)	Smart narration & scene description
Backend Language	Python 3.10+	Core pipeline orchestration
Config Management	YAML + argparse	Runtime settings, thresholds, language
Async Task	asyncio / threading	Non-blocking TTS & API calls

3. System Modules — Detailed

3.1 Image Acquisition

- OpenCV captures frames from default webcam (index 0) at ~30fps
- Each frame resized to 640×480 before entering the AI pipeline
- Lightweight, energy-efficient processing loop
- Frame buffer prevents pipeline stalls during heavy inference

3.2 Object Detection (YOLOv8)

- Model: YOLOv8n for speed (prototype), YOLOv8s for better accuracy
- Confidence threshold: ≥ 0.80 (all lower detections discarded)
- Targeted classes: person, chair, table, car, bicycle, door, stairs, bench, pole, wall
- Output per object: bounding box (x1,y1,x2,y2), class label, confidence score
- Processes one frame at a time in real-time loop

3.3 Depth Estimation (MiDaS)

- Model: MiDaS v2.1 Small — fast inference on CPU (20-40ms)
- Input: same 640×480 frame as YOLO detection
- Output: relative depth map (normalized 0.0–1.0 per pixel)
- Distance calc: sample depth value at center of each bounding box
- Calibration: map relative depth to approximate meters (0.5–5.0m range)
- Higher depth value = closer to camera

3.4 Direction Estimation

- Camera frame divided into 3 equal vertical zones (33% each)
- Zone 0–33%: LEFT | Zone 33–66%: CENTER | Zone 66–100%: RIGHT
- Object x-center coordinate of bounding box determines zone
- Output label: LEFT / CENTER / RIGHT per detected object

3.5 Object Tracking (ByteTrack)

- Assigns unique persistent Track ID to each detected object
- Tracks objects across frames using IoU + Kalman filter prediction
- Alert suppression: same Track ID not re-announced within 3 seconds
- Re-announces if: object moves to new zone, distance changes >30%, new object enters

- Prevents repetitive audio spam that would confuse/disorient user

3.6 Decision Engine

- Distance < 0.8m (any direction): URGENT — 'Stop immediately, obstacle very close'
- Distance 0.8–1.5m (CENTER): WARNING — 'Obstacle ahead, slow down'
- Distance 1.5–2.5m: INFO — 'Chair on your left', 'Person ahead'
- Distance > 2.5m: SILENT (no announcement unless approaching fast)
- Max 2 simultaneous alerts — closest + most centered objects prioritized
- Rule-based logic only — no LLM in critical navigation decision path

3.7 Audio Feedback System

- Language toggle: English / Hindi set at startup (or via keyboard shortcut 'H')
- English: 'Person ahead', 'Chair on your left', 'Wall very close, stop'
- Hindi: 'Aage insaan hai', 'Bayi taraf kursi hai', 'Diwar bahut paas hai, ruko'
- Async TTS runs in separate thread — does NOT block the main vision loop
- Earphones or bone-conduction headphones recommended for real use
- Audio queue prevents overlapping speech instructions

3.8 Claude claude-opus-4-6 Integration

- NOT used in the real-time critical navigation path (too slow for 30fps loop)
- Triggered on: user voice command 'describe surroundings', keyboard press 'D', or every 30s
- Input: base64-encoded annotated frame + detection JSON
- Output: 2-3 sentence natural language scene description in English or Hindi
- Also used for: smart alert generation, complex multi-object narration, context-aware advice
- API call is async — response delivered via TTS when ready (1-3 second delay acceptable)

4. Hallucination Prevention Strategy

For a navigation assistant serving blind users, incorrect information is NOT just an inconvenience — it can cause physical harm. Every layer of the system is designed to minimize false detections and erroneous voice instructions.

Technique	Implementation Detail	Threshold / Rule
Confidence Thresholding	YOLOv8 detections below threshold are silently discarded	$\geq 80\%$ confidence
Multi-Frame Verification	Object must appear in N consecutive frames before announcement	3 consecutive frames
Tracking Validation	ByteTrack ID must be stable (not flickering)	5+ tracked frames
Distance Stability	Estimated distance must not fluctuate wildly between frames	$\leq 20\%$ variance
Rule-Based Decisions	Navigation alerts use hardcoded rules, never LLM generation	Always enforced
Claude API Guard	Claude used ONLY for scene description, never for 'stop/go' decisions	Non-critical path only
Bounding Box Size	Very tiny bounding boxes ($< 1\%$ frame area) ignored as noise	$< 1\%$ frame area = skip
Class Filtering	Only pre-approved class list triggers alerts (no random objects)	Whitelist enforced

5. Performance Requirements

Pipeline Stage	Target Latency	Implementation Notes
Webcam Frame Capture	~10ms	OpenCV default @ 30fps
YOLOv8n Inference (CPU)	15–30ms	YOLOv8n on modern laptop CPU
MiDaS Depth Inference	20–40ms	MiDaS Small, CPU optimized
ByteTrack Update	~5ms	Lightweight IoU + Kalman
Decision Engine	< 2ms	Rule-based, near instant
TTS Generation	20–50ms	Async thread, non-blocking
Total End-to-End	50–130ms	Target < 100ms, max 130ms

Note: YOLOv8 and MiDaS can run in parallel threads to reduce total latency. On a modern laptop with a dedicated GPU, YOLOv8 drops to ~5ms and MiDaS to ~8ms.

6. Project Folder Structure

```
ai-nav-assistant/  
  
  main.py                # Entry point – starts the full pipeline  
  config.yaml            # Thresholds, language, model paths, TTS settings  
  requirements.txt  
  README.md  
  modules/  
    detector.py          # YOLOv8 wrapper – load model, run inference  
    depth_estimator.py   # MiDaS wrapper – load model, compute depth map  
    tracker.py           # ByteTrack integration – assign + maintain track I  
Ds  
    direction.py         # LEFT / CENTER / RIGHT zone computation  
    decision_engine.py   # Priority logic, alert rules, distance thresholds  
    voice.py             # TTS engine (pyttsx3 EN + gTTS HI) with async queu  
e  
    claude_client.py     # Claude API wrapper – scene description calls  
  models/  
    yolo8n.pt            # Downloaded on first run (Ultralytics auto-downloa  
d)  
    midas_v21_small.pt   # Downloaded from PyTorch Hub  
  utils/  
    frame_utils.py       # Resize, annotate, crop utilities  
    logger.py            # Structured logging for debug  
    calibration.py       # Depth-to-meters mapping calibration
```

7. Claude claude-opus-4-6 Prompt Engineering Guide

All prompts below are production-ready for claude-opus-4-6. They are copy-paste ready. Variables in {curly_braces} must be replaced at runtime with actual values. Section 8 contains a complete variable reference table.

7.1 System Prompt — Core Navigation Assistant

Prompt 7.1 — System Prompt — Core Navigation Assistant

You are an AI navigation assistant for visually impaired users. You receive camera frames and object detection data from a real-time vision system.

Your role is to generate clear, concise, and accurate voice instructions to help the user navigate safely.

Rules:

- Only describe objects CONFIRMED by the detection system – do NOT hallucinate
- Prioritize by proximity: closest objects mentioned first
- Keep responses SHORT – maximum 2 sentences
- Use simple, direct, everyday language
- Respond in user's preferred language: {language}
- NEVER guess or assume objects not present in detection data
- If detection data is empty or uncertain, say: "Path seems clear"
- For urgent obstacles (< 1m), start with "STOP" or "RUKO" (Hindi)

Detection input format:

```
{ "objects": [ { "label": "chair", "distance_m": 1.2, "direction": "LEFT", "confidence": 0.91 } ] }
```

Output: Plain text voice instruction ONLY. No JSON, no markdown, no explanation.

7.2 Scene Understanding — Full Environment Description

Prompt 7.2 — Scene Understanding — Full Environment Description

You are a scene describer for a visually impaired navigation assistant.

You will receive:

1. A base64-encoded camera frame (annotated with bounding boxes)
2. A JSON list of all detected objects with distances and directions

Your task: Generate a natural, helpful 2-3 sentence description of the user's current environment.

Rules:

- Mention the most important obstacles FIRST (closest, most dangerous)
- Include approximate distances in meters
- Explicitly mention if the path directly ahead is clear or blocked
- Use conversational, friendly language – not technical terms
- Language: {language}
- Do NOT mention confidence scores, model names, or bounding boxes
- Do NOT say "I can see" or "The image shows" – speak directly to the user

Image: {base64_frame}

Detections: {detections_json}

Output: Natural language scene description. No markdown, no bullet points.

7.3 Hindi Translation — Navigation Instruction

Prompt 7.3 — Hindi Translation — Navigation Instruction

Translate the following English navigation instruction into natural, conversational Hindi (Devanagari script).

Rules:

- Maximum 1 sentence – keep it SHORT
- Use simple everyday Hindi that any person can understand
- Preserve urgency: if the original sounds urgent, your Hindi must also sound urgent
- Do NOT add extra words, explanations, or context
- Do NOT transliterate (write in Devanagari script, not Roman script)

- If the instruction contains a direction like "left" → "■■■■", "right" → "■■■■", "ahead" → "■■■"

English instruction to translate: {english_instruction}

Output: Hindi translation ONLY. Nothing else.

7.4 Emergency Stop Alert — Critical Proximity

Prompt 7.4 — Emergency Stop Alert — Critical Proximity

CRITICAL SAFETY SITUATION: An obstacle has been detected extremely close to the user.

Distance: {distance} meters

Object: {object_label}

Direction: {direction}

Language: {language}

Generate an URGENT stop instruction.

Requirements:

- MUST convey immediate danger clearly
- MUST instruct the user to STOP immediately
- Maximum 6 words
- Language: {language}
- English examples: "STOP! Wall directly ahead!", "DANGER! Person very close!"
- Hindi examples: "■■■■! ■■■■ ■■■■ ■■!", "■■■■! ■■■ ■■■■ ■■!"

Output: Urgent instruction ONLY. No explanation.

7.5 Multi-Object — Prioritized Alert Generation

Prompt 7.5 — Multi-Object — Prioritized Alert Generation

Multiple objects have been detected simultaneously. Generate ONE combined voice instruction covering the 2 most dangerous objects.

Priority rules (apply in order):

1. Objects closer than 1m get HIGHEST priority regardless of direction
2. CENTER direction > LEFT/RIGHT if same distance
3. Objects labeled "person" or "car" get +0.5 priority boost (dynamic obstacles)
4. Maximum 2 objects mentioned in a single instruction

Detections JSON: {detections_json}

Language: {language}

Output: Single voice instruction, maximum 12 words. Plain text only.

7.6 Stair Detection — Safety Alert

Prompt 7.6 — Stair Detection — Safety Alert

A staircase has been detected in the user's path.

Stair direction: {stair_direction} (UP / DOWN / UNKNOWN)

Distance to stairs: {distance} meters

Language: {language}

Generate a safety instruction that:

- Clearly warns about stairs ahead
- Mentions UP or DOWN direction if known
- Advises the user to slow down and be careful
- Sounds calm but clear – not panicked
- Maximum 10 words

Output: Voice instruction only.

7.7 Clear Path — Reassurance Message

Prompt 7.7 — Clear Path — Reassurance Message

7.9 Dynamic Instruction Generator — Context-Aware

Prompt 7.9 — Dynamic Instruction Generator — Context-Aware

You are an adaptive navigation instructor. Based on the user's movement history and current detections, generate the most useful next instruction.

Current detections: {detections_json}

Recent movement history (last 5 seconds): {movement_history}

User's stated destination: {destination}

Language: {language}

Rules:

- If user is approaching an obstacle: warn them NOW
- If user just avoided an obstacle: optionally confirm ("Good, obstacle cleared")
- If path ahead is open: give positive directional guidance toward destination
- If user has been stationary for 5+ seconds: give a gentle prompt
- Keep instruction under 12 words

Output: Context-aware navigation instruction. Plain text only.

7.10 Object Distance Narration — Detailed Report

Prompt 7.10 — Object Distance Narration — Detailed Report

Generate a detailed distance narration for all detected objects in the scene.

Detections: {detections_json}

Language: {language}

For each object, naturally describe:

- What the object is
- Approximately how far it is (in meters, rounded to 0.5m)
- Which direction (left, right, ahead)

Format as a natural spoken paragraph, not a list.

Maximum 3 sentences.

Language: {language}

Output: Spoken distance report. Natural language, no markdown.

7.11 Environment Type Classifier — Context Detection

Prompt 7.11 — Environment Type Classifier — Context Detection

Based on the objects detected in this scene, classify the environment type and provide context-appropriate navigation advice.

Detections: {detections_json}

Optional frame description: {frame_description}

Language: {language}

Environment types to consider:

- Indoor (office / home / corridor / room)
- Outdoor (road / sidewalk / park)
- Public space (mall / station / cafeteria)
- Stairwell / elevator area
- Unknown

Output format (plain text, 2 sentences):

Sentence 1: Environment classification

Sentence 2: One navigation tip appropriate for this environment type

Language: {language}

7.12 Danger Level Assessor — Risk Scoring

Prompt 7.12 — Danger Level Assessor — Risk Scoring

Assess the overall danger level of the current scene for a visually impaired person and recommend immediate action.

Detections: {detections_json}

User speed estimate: {user_speed} (stationary / slow / fast)

Language: {language}

Danger levels:

- SAFE: No objects within 2m, path clear
- CAUTION: Objects 1-2m away, no immediate threat
- WARNING: Object within 1m, user should slow down
- DANGER: Object within 0.8m, user must stop

Output (2 parts, plain text):

1. Danger level: [SAFE / CAUTION / WARNING / DANGER]
2. Recommended voice instruction in {language} (max 8 words)

8. Prompt Variables Reference

Variable	Type	Example Value	Used In Prompts
{language}	string	English / Hindi	All prompts
{detections_json}	JSON string	[{"label":"chair","distance_m":1.2,"direction":"LEFT","confidence":0.91}]	7.1, 7.5, 7.10, 7.11, 7.12
{base64_frame}	base64 string	/9j/4AAQSkZJRgAB...	7.2
{english_instruction}	string	Chair on your left	7.3
{object_label}	string	wall / person / chair	7.4
{distance}	float (meters)	0.7 / 1.2 / 2.5	7.4, 7.6
{direction}	string	LEFT / CENTER / RIGHT	7.4, 7.6
{stair_direction}	string	UP / DOWN / UNKNOWN	7.6
{clear_seconds}	int	5 / 10 / 30	7.7
{movement_history}	string	moved_forward, paused, turned_left	7.9
{destination}	string	exit door / kitchen / street	7.9
{frame_description}	string	Indoor hallway with fluorescent lights	7.11
{user_speed}	string	stationary / slow / fast	7.12

9. Development Phases

Phase	Goal	Key Deliverable	Est. Time
Phase 1	Basic detection + voice	YOLOv8 + pyttsx3 on webcam, English voice working	2-3 days
Phase 2	Add depth estimation	MiDaS integrated, distance output per object	2-3 days
Phase 3	Tracking + decision engine	ByteTrack, no repeat alerts, priority rules active	2-3 days
Phase 4	Hindi bilingual toggle	gTTS Hindi working, keyboard toggle EN/Hi	1-2 days
Phase 5	Claude API integration	Scene description on demand, async response	2 days
Phase 6	Hallucination hardening	Multi-frame verify, confidence gates, bounding box filter	1-2 days
Phase 7	Polish + portfolio demo	README, demo video, GitHub push, live demo	2 days

10. Python Requirements (requirements.txt)

```
# Core AI / Vision
ultralytics>=8.0.0          # YOLOv8 + ByteTrack
opencv-python>=4.8.0        # Webcam capture + frame processing
torch>=2.0.0                 # PyTorch – MiDaS backbone
torchvision>=0.15.0
timm>=0.9.0                  # MiDaS model backbone (ViT)

# Depth Estimation
# MiDaS loaded via torch.hub – no pip install needed

# Anthropic Claude API
anthropic>=0.20.0

# Text-to-Speech
pyttsx3>=2.90                # Offline English TTS
gTTS>=2.3.2                  # Hindi TTS (requires internet)
playsound>=1.3.0             # Audio playback for gTTS output
pygame>=2.5.0                # Alternative audio playback

# Utilities
numpy>=1.24.0
Pillow>=10.0.0
pyyaml>=6.0                  # Config file parsing
python-dotenv>=1.0.0         # .env for API keys
requests>=2.31.0

# Optional: for async TTS
edge-tts>=6.1.9              # Microsoft Edge TTS (better voice quality)
asyncio                       # Built-in Python, no install needed
```

AI Navigation Assistant — PRD v1.0 | Built for Laptop + Webcam Prototype | English + Hindi | claude-opus-4-6